Simulação e Análise de Arquitetura Interna de um Computador

Parte Teórica

João Pedro Vieira da Costa Chaves — UniFECAF — 2026

Sumário

1. Introdução e Contextualização da Solução Proposta para a NeoCore Tech
2. Descrição, Fundamentação e Justificativa da Arquitetura Escolhida
   - 2.1. Modelo de Von Neumann com Acumulador e Arquitetura Harvard Modificada
   - 2.2. Convergência entre Filosofias CISC e RISC (Micro-operações e RTL)
   - 2.3. Limites Físicos, Eficiência Energética e TI Verde
   - 2.4. Objetivo e Justificativa do Recorte Didático de 8 Bits da PoC
3. Diagrama Geral da Arquitetura
4. Componentes da CPU, Barramentos e Hierarquia de Memória
   - 4.1. Unidade Lógica e Aritmética (ULA)
   - 4.2. Unidade de Controle (UC)
   - 4.3. Registradores Internos da PoC (PC, MAR, MBR, IR, AC e SAIDA)
   - 4.4. Barramentos de Endereços, Dados e Controle
   - 4.5. Hierarquia de Memória e o Papel da Memória Cache
5. O Ciclo de Instrução (Fetch – Decode – Execute) e Caso de Teste
   - 5.1. Fase de Busca (Fetch)
   - 5.2. Fase de Decodificação (Decode) e Responsabilidade na PoC
   - 5.3. Fase de Execução (Execute)
   - 5.4. Caso de Teste Passo a Passo e Tabela de Trace (Soma 5 + 3 = 8)
6. Modelagem e Implementação da Parte Prática
   - 6.1. Ferramentas Utilizadas e Arquitetura do Projeto
   - 6.2. Estrutura do Circuito e Fluxo de Dados
   - 6.3. Recursos Didáticos Interativos do Simulador Web
7. Conclusões e Limitações
   - 7.1. Síntese Coordenada do Ciclo de Instrução e Resposta
   - 7.2. Limitações da Prova de Conceito (PoC)
8. Referências

1. Introdução e Contextualização da Solução Proposta para a NeoCore Tech

No ecossistema atual de desenvolvimento de software e ensino de engenharia e Gestão da Tecnologia da Informação, observa-se uma lacuna crescente entre a escrita de código de alto nível e a compreensão aprofundada da execução física no hardware. Embora estudantes e desenvolvedores projetem sistemas complexos com grande agilidade, frequentemente impulsionados por camadas de abstração e ferramentas automatizadas, a dinâmica interna do processador permanece velada. Conceitos fundamentais — como a orquestração da Unidade Central de Processamento (CPU), a cooperação entre a Unidade Lógica e Aritmética (ULA) e a Unidade de Controle (UC), a retenção síncrona em registradores de alta velocidade, a hierarquia de memória e a sinalização em barramentos — acabam abordados de maneira puramente teórica ou compartimentada, dificultando a assimilação sistêmica do funcionamento da máquina (STALLINGS, 2010).

Visando solucionar esse desafio pedagógico, este trabalho apresenta uma proposta desenvolvida no contexto da NeoCore Tech, empresa fictícia especializada em soluções educacionais e laboratoriais em sistemas computacionais: a concepção, modelagem e implementação de uma Prova de Conceito (Proof of Concept — PoC) denominada NeoCore 8-bit CPU. O objetivo central do projeto é demonstrar, de forma visual, estruturada e funcional, a integração entre o projeto lógico de um computador e sua concretização em engenharia digital, articulando dois domínios que se complementam conceitualmente (STALLINGS, 2010; TANENBAUM; AUSTIN, 2013):

- **Arquitetura de Computadores**: refere-se aos atributos de um sistema visíveis ao programador de baixo nível e que possuem impacto direto sobre a execução lógica de um programa, tais como o conjunto de instruções (Instruction Set Architecture — ISA), o número de bits utilizados para representar tipos de dados, os mecanismos de Entrada e Saída (E/S) e as técnicas de endereçamento de memória;

- **Organização de Computadores**: refere-se às unidades operacionais e às suas interconexões físicas que materializam as especificações arquiteturais, abrangendo os sinais de controle, as interfaces entre o processador e os periféricos, a topologia interna dos barramentos e a tecnologia de memória empregada.

Para tornar esses dois domínios diretamente observáveis sem a opacidade dos processadores comerciais de múltiplos gigahertz (GHz), a solução proposta para a NeoCore Tech foi construída em duas frentes integradas e coerentes entre si:

1. **Circuito digital esquemático (src/neocore_cpu.circ)**: projetado e validado no simulador lógico Logisim-Evolution v5.0.0, operando com palavras de 8 bits sob frequência de clock de 1,0 Hz, no qual os cinco sinais primários de habilitação do caminho de dados são acionados explicitamente para evidenciar cada transferência entre registradores (Register Transfer Level — RTL);
2. **Simulador interativo web (index.html, src/styles.css e src/script.js)**: aplicação

front-end independente que reproduz fielmente a topologia do circuito digital em um diagrama vetorial (SVG) interativo, preservando o Modo Manual idêntico ao esquemático do Logisim-Evolution (no qual o estudante atua como Unidade de Controle) e incorporando um Modo Automático como extensão pedagógica em software (dotado de Unidade de Controle baseada em máquina de estados finitos, conjunto de instruções de 8 bits e simulação de Memória Cache L1 de mapeamento direto).
2. **Simulador interativo web (index.html, src/styles.css e src/script.js)**: aplicação front-end independente que reproduz fielmente a topologia do circuito digital em um diagrama vetorial (SVG) interativo, preservando o Modo Manual idêntico ao esquemático do Logisim-Evolution (no qual o estudante atua como Unidade de Controle) e incorporando um Modo Automático como extensão pedagógica em software (dotado de Unidade de Controle baseada em máquina de estados finitos, conjunto de instruções de 8 bits e simulação de Memória Cache L1 de mapeamento direto).

2. Descrição, Fundamentação e Justificativa da Arquitetura Escolhida

Como referência de contexto industrial para o estudo de Arquitetura e Organização de Computadores, observa-se a evolução dos processadores comerciais modernos, como a família x86-64 (exemplificada pelas linhas Intel Core e AMD Ryzen) e arquiteturas ARM. Embora o modelo prático desenvolvido neste trabalho adote uma estrutura enxuta de 8 bits voltada ao ensino dos fundamentos clássicos, analisar como os processadores contemporâneos estruturam seus barramentos e memórias permite justificar com precisão as escolhas arquiteturais da PoC (Prova de Conceito).

2.1. Modelo de Von Neumann com Acumulador e Arquitetura Harvard Modificada

A arquitetura selecionada para a PoC fundamenta-se no **modelo clássico de Von Neumann (ou arquitetura de Princeton)**, formulado por John von Neumann em 1945, no relatório preliminar sobre o computador EDVAC, e desenvolvido com colaboradores a partir de 1946 no projeto do computador IAS (STALLINGS, 2010; TANENBAUM; AUSTIN, 2013), combinado com uma organização interna baseada em acumulador de barramentos explícitos, inspirada no computador didático SAP-1 (Simple-As-Possible) (MALVINO; BROWN, 1992).

No modelo de Von Neumann, verificam-se três princípios estruturais que regem integralmente a PoC implementada:

1. **Memória principal unificada:** instruções de programa e dados compartilham o mesmo espaço de endereçamento linear na memória principal (RAM 256 x 8 bits, endereços 00h a FFh) e utilizam um único caminho físico de comunicação com a CPU. Uma palavra de 8

bits armazenada na RAM pode ser tratada como instrução (quando encaminhada ao Registrador de Instrução IR) ou como operando numérico (quando operada pela ULA);

2. **Busca sequencial controlada por Contador de Programa:** na ausência de desvios, o Contador de Programa (PC) avança sequencialmente (PC ← PC + 1), apontando para o endereço seguinte da memória;

3. **Caminho de dados centralizado no Acumulador (AC):** em uma organização de acumulador (arquitetura de um endereço), a ULA mantém uma de suas entradas permanentemente conectada ao registrador AC, enquanto a segunda entrada recebe o operando lido da memória através do Registrador de Buffer de Memória (MBR), gravando o resultado de volta no próprio AC (AC ← AC + MBR).

Contudo, a arquitetura de Von Neumann pura apresenta uma limitação física de vazão conhecida como **Gargalo de Von Neumann (Von Neumann Bottleneck)**. Como a busca de uma instrução e a leitura ou escrita de um operando de dados não podem ocorrer simultaneamente pelo barramento compartilhado, a CPU frequentemente aguarda o tráfego de informações (wait state), efeito agravado pela discrepância histórica entre a alta velocidade do processador e a maior latência da memória principal (Memory Wall).

Em contraposição, a **arquitetura Harvard** pura separa fisicamente a memória de instruções da memória de dados em barramentos independentes, sendo comum em microcontroladores e processadores digitais de sinais (DSPs). Já os processadores modernos de uso geral adotam uma solução híbrida denominada Arquitetura Harvard Modificada (TANENBAUM; AUSTIN, 2013):

- **Externamente (Memória Principal RAM e Caches L2/L3)**: preserva-se a unificação de Von Neumann, mantendo código e dados no mesmo espaço de endereçamento para simplificar o carregamento de programas pelo sistema operacional;

- **Internamente (Cache L1)**: adota-se a divisão física da arquitetura Harvard, segmentando o primeiro nível de cache em L1i (instruções) e L1d (dados), permitindo buscar uma instrução e acessar um dado no mesmo ciclo de clock.

Para o escopo laboratorial da NeoCore Tech, escolheu-se a arquitetura de Von Neumann pura com acumulador porque a duplicação física de memórias e barramentos do modelo Harvard aumentaria a complexidade visual do esquemático sem agregar clareza ao objetivo central: demonstrar como um endereço parte do PC, acessa a RAM via MAR e entrega a palavra ao MBR.

2.2. Convergência entre Filosofias CISC e RISC (Micro-operações e RTL)

No âmbito do conjunto de instruções (Instruction Set Architecture — ISA), distinguem-se duas filosofias clássicas de projeto (STALLINGS, 2010; PATTERSON; HENNESSY, 2017):

- **CISC (Complex Instruction Set Computer)**: caracteriza-se por instruções de tamanho variável e alta densidade semântica, capazes de combinar acesso direto à memória e operação aritmética em uma única instrução, exigindo unidades de controle microprogramadas mais complexas;

- **RISC (Reduced Instruction Set Computer)**: adota instruções de tamanho fixo, formato regular para decodificação direta em hardware (hardwired control) e arquitetura load/store, na qual operações aritméticas ocorrem estritamente entre registradores.

Processadores x86-64 contemporâneos convergem ambas as abordagens: mantêm uma interface externa CISC para compatibilidade de software, mas utilizam decodificadores internos que traduzem instruções complexas em sequências de operações internas simples e de largura fixa denominadas micro-ops (μops), executadas em pipelines superescalares de estilo RISC (TANENBAUM; AUSTIN, 2013).

Para fins de rigor conceitual neste relatório, distingue-se o conceito de μops internas de processadores comerciais das micro-operações de transferência entre registradores (RTL — Register Transfer Level). As micro-operações RTL representam os passos elementares sincronizados por cada pulso de clock (T0, T1, T2, T3) durante as fases de busca, decodificação e execução, constituindo o objeto direto de observação na PoC desenvolvida.

2.3. Limites Físicos, Eficiência Energética e TI Verde

A evolução da arquitetura de computadores também é condicionada por restrições físicas de dissipação térmica e consumo energético em circuitos digitais CMOS (Complementary Metal-Oxide-Semiconductor). A potência dinâmica (P) dissipada por um processador é expressa aproximadamente pela equação:

$$P \approx \alpha \cdot C \cdot V^2 \cdot f$$

Na expressão acima, α representa o fator de atividade de chaveamento das portas lógicas, C é a capacitância total comutada do circuito (proporcional à área de silício e à quantidade de transistores), V corresponde à tensão de alimentação e f denota a frequência de operação do clock (STALLINGS, 2010; PATTERSON; HENNESSY, 2017). Conforme analisado por Tanenbaum e Bos (2016) e por Stallings (2010), as barreiras térmicas (Power Wall) impediram a continuidade do aumento linear da frequência f, levando a indústria a adotar arquiteturas com múltiplos núcleos (multicore) e controle rigoroso de capacitância e tensão, em consonância com as diretrizes de TI Verde voltadas à redução do consumo elétrico e da demanda de refrigeração em data centers.

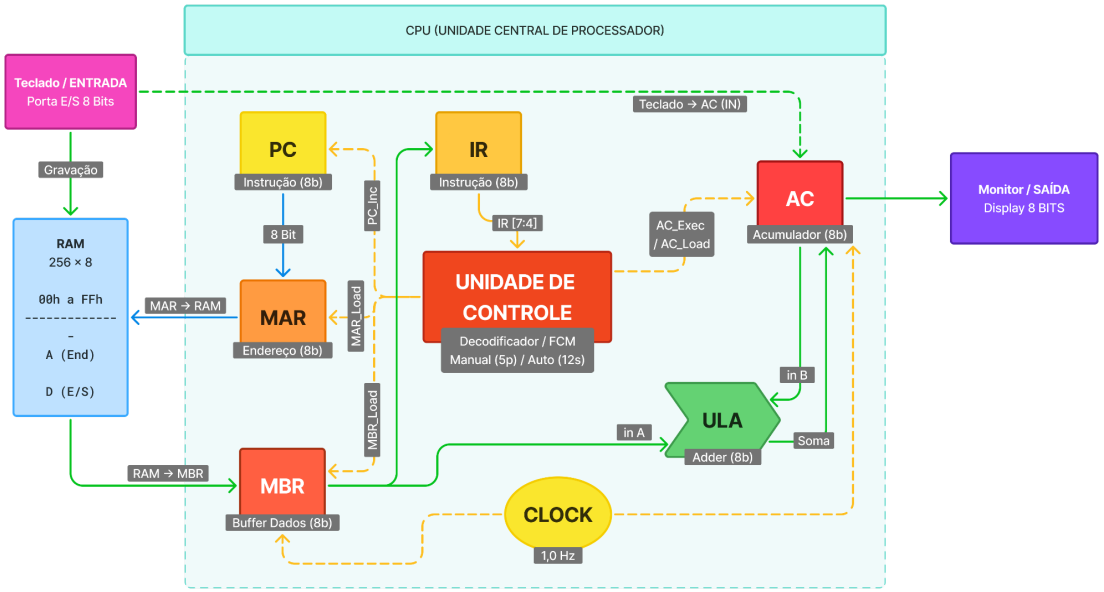
2.4. Objetivo e Justificativa do Recorte Didático de 8 Bits da PoC

Enquanto processadores comerciais integram bilhões de transistores, múltiplos núcleos e pipelines profundos, tais estruturas inviabilizam a inspeção visual direta do caminho de dados por um estudante. Por essa razão, a PoC NeoCore 8-bit CPU adota deliberadamente um recorte didático de 8 bits operando a 1,0 Hz, justificado por três critérios pedagógicos e de engenharia:

1. Legibilidade binária e hexadecimal imediata: uma palavra de 8 bits (1 byte) é representada exatamente por dois dígitos hexadecimais (00h a FFh) e compreende o intervalo inteiro sem sinal de 0 a 255, permitindo que o estudante verifique manualmente, bit a bit, cada adição na ULA;
2. Espaço de endereçamento completo e inspecionável: um barramento de endereços de 8 bits endereça exatamente 2^8 = 256 posições de memória (RAM 256 x 8), capacidade suficiente para alojar instruções e operandos mantendo visualização integral em páginas de 16 bytes;
3. Transparência do caminho de dados (datapath): no circuito base do Logisim-Evolution, foram omitidos pipelines e múltiplos núcleos para focar no acionamento explícito dos sinais de habilitação a 1,0 Hz, complementado no simulador web pela possibilidade de alternar entre o Modo Manual (fiel ao esquemático .circ) e o Modo Automático (com Unidade de Controle e Cache L1 didática).

3. Diagrama Geral da Arquitetura

A estrutura global da arquitetura proposta e implementada para a NeoCore Tech é apresentada na Figura 1, contemplando a Unidade Central de Processamento (CPU), os registradores internos (PC, MAR, MBR, IR, AC), a Unidade Lógica e Aritmética (ULA), a Unidade de Controle (UC / linhas de controle), a memória principal (RAM 256 x 8), a representação da memória Cache L1, os dispositivos de Entrada (Teclado) e Saída (Monitor / SAIDA) e os barramentos de dados, endereços e controle.



Legenda:Diagrama de blocos da arquitetura NeoCore 8-bit CPU baseada no modelo de Von Neumann com organização de acumulador. As linhas contínuas azuis representam o barramento de endereços (8 bits, unidirecional: PC → MAR → RAM), as linhas contínuas verdes indicam o barramento e os caminhos internos de dados (8 bits: RAM ↔ MBR, MBR → IR, MBR → ULA, AC → ULA, ULA → AC e AC → Monitor/SAIDA e Teclado → AC/RAM), e as linhas tracejadas laranjas/vermelhas representam o barramento de controle e a rede de clock síncrono (1,0 Hz).

Para assegurar total coerência entre este relatório teórico e os artefatos construídos na prática, cada bloco funcional da Figura 1 classifica-se da seguinte forma quanto ao seu estado de implementação:

- Blocos implementados tanto no circuito digital (src/neocore_cpu.circ) quanto no Simulador Web (index.html — Modo Manual e Modo Automático):
  - Registradores internos de 8 bits (PC, MAR, MBR, IR e AC): construídos com flip-flops tipo D sensíveis à borda de subida do clock e controlados por sinais de habilitação de escrita (Write Enable — WE);

  - Unidade Lógica e Aritmética (ULA — Adder de 8 bits): somador binário combinacional de 8 bits que calcula continuamente AC + MBR;

  - Memória Principal (RAM 256 x 8): matriz de memória volátil de 256 posições de 8 bits (00h a FFh);

  - Dispositivos de Entrada e Saída (Teclado e Monitor): pino de entrada de 8 bits (Teclado) e visor binário/numérico de saída de 8 bits (Monitor) acoplado à saída do Acumulador;

  - Rede de CLOCK (1,0 Hz) e Barramentos de Endereços (8 bits), Dados (8 bits) e Controle Manual (5 linhas: MAR_Load, RAM_Read_MBR, IR_Load, PC_Inc e AC_Exec).

- Blocos implementados exclusivamente como extensão em software no Simulador Web (index.html / src/script.js — Modo Automático):
  - Unidade de Controle Automática (Decoder UC / FSM UC_TABLE): máquina de estados finitos que decodifica IR[7:4] (opcode de 4 bits) e IR[3:0] (operando/endereço de 4 bits) e comanda automaticamente 12 sinais de controle no painel HUD do SVG; no arquivo neocore_cpu.circ, o bloco Decoder UC é apenas uma moldura visual (sem portas de decodificação interna), pois o controle no Logisim é exercido manualmente pelo estudante;
  - Memória Cache L1 Didática (Cache 4L × 2B): módulo de mapeamento direto interposto entre MAR/MBR e a RAM 256 x 8 no simulador web para demonstração prática de cache hit e cache miss; no arquivo neocore_cpu.circ, esse nível da hierarquia é conceitual;
  - Memória Cache L1 Didática (Cache 4L × 2B): módulo de mapeamento direto interposto entre MAR/MBR e a RAM 256 x 8 no simulador web para demonstração prática de cache hit e cache miss; no arquivo neocorecpu.circ, esse nível da hierarquia é conceitual;

  - Registrador de Saída Dedicado (SAIDA / OUT): registrador de 8 bits atualizado pela

instrução OUT (0x5) no Modo Automático do simulador web.
- Blocos puramente conceituais (discutidos na fundamentação teórica, não implementados na PoC):

  - Memórias Cache multinível (L2 e L3), armazenamento secundário não volátil (SSD/HD), controlador de interrupções, acesso direto à memória (DMA) e instruções de desvio condicional baseadas em flags.

4. Componentes da CPU, Barramentos e Hierarquia de Memória

A Unidade Central de Processamento (CPU) constitui o subsistema encarregado de buscar instruções na memória principal, examinar sua codificação binária e coordenar a execução sequencial das micro-operações sobre os dados (STALLINGS, 2010). A seguir, detalham-se a função teórica e a implementação efetiva de cada componente na PoC.

4.1. Unidade Lógica e Aritmética (ULA)

Sob o prisma teórico, a ULA é o circuito combinacional responsável por executar operações aritméticas elementares (adição, subtração, incremento), testes relacionais booleanos (A > B, A = B, A \neq B) e operações lógicas bit a bit (AND, OR, XOR, NOT e deslocamentos) sobre os operandos fornecidos pelos registradores, além de atualizar indicadores de estado (flags de Zero, Carry, Overflow e Sign) (TANENBAUM; AUSTIN, 2013).

- **Implementação na PoC**: No circuito neocore_cpu.circ e no Modo Manual do simulador web, a ULA foi implementada de forma enxuta como um somador binário fixo de 8 bits (Adder), sem linhas seletoras de operação lógica e sem registrador de flags. Sua entrada A (superior) recebe continuamente os 8 bits provenientes do MBR, enquanto sua entrada B (inferior) recebe a realimentação de 8 bits do Acumulador (AC), produzindo combinacionalmente a soma em aritmética modular de 8 bits:

$$Saída_{ULA} = (AC + MBR) mod\ 256$$

O resultado presente na saída da ULA somente é retido no Acumulador (AC) quando o sinal de controle AC_Exec (ou AC_Load no Modo Automático) encontra-se ativo durante a borda de subida do CLOCK. No Modo Automático do simulador web, as operações sobre o Acumulador seguem a semântica da tabela UC_TABLE e os sinais ALU_Op (ADD e PASS):

- **As instruções de operando imediato operam diretamente sobre o nibble inferior IR[3:0]**: ADDI calcula AC ← AC + IR[3:0] e LDI realiza a carga direta AC ← IR[3:0];
- **Apenas as instruções de acesso à memória RAM utilizam o MBR**: LDA transfere o dado

lido para o acumulador (AC ← MBR, com ALU_Op = PASS), enquanto ADD soma o dado da memória ao acumulador (AC ← AC + MBR, com ALU_Op = ADD);
- **Apenas as instruções de acesso à memória RAM utilizam o MBR**: LDA transfere o dado lido para o acumulador (AC ← MBR, com ALUOp = PASS), enquanto ADD soma o dado da memória ao acumulador (AC ← AC + MBR, com ALUOp = ADD);

- A instrução IN transfere o byte do dispositivo de entrada diretamente ao acumulador (AC ← TECLADO), sem passar pelo MBR, enquanto STA transfere o acumulador para o MBR (MBR ← AC) para posterior gravação na memória (RAM[MAR] ← MBR).

4.2. Unidade de Controle (UC)

Teoricamente, a Unidade de Controle é o subsistema sequenciador que coordena o funcionamento do processador: ela recebe os pulsos de clock, interpreta o código de operação (opcode) contido no Registrador de Instrução (IR) e emite, na sequência temporal exata, os sinais de controle que habilitam a leitura e a escrita nos registradores, selecionam a operação da ULA e comandam os acessos à memória principal (STALLINGS, 2010).

- Implementação na PoC: A função da Unidade de Controle manifesta-se de duas formas complementares no projeto:

1. **No circuito neocore_cpu.circ e no Modo Manual do simulador web**: a função da UC é exercida manualmente pelo próprio estudante, por meio do acionamento de cinco pinos de controle discretos (MAR_Load, RAM_Read_MBR, IR_Load, PC_Inc e AC_Exec), permitindo vivenciar na prática o papel de coordenação temporal de cada micro-operação;

2. **No Modo Automático do simulador web (src/script.js)**: a função da UC é executada por uma máquina de estados finitos orientada por tabela (UC_TABLE), que percorre automaticamente os estados T0–T4 (nas instruções ADDI, LDI, OUT, IN e HLT) ou T0–T6 (nas instruções LDA, ADD e STA), conforme a instrução executada, decodifica o nibble superior IR[7:4] no estado T3 e aciona de maneira autônoma os 12 sinais do barramento de controle.

4.3. Registradores Internos da PoC (PC, MAR, MBR, IR, AC e SAIDA)

Os registradores internos constituem memórias estáticas de altíssima velocidade situadas dentro da CPU, construídas com flip-flops para reter endereços, instruções e operandos durante o ciclo de máquina (TANENBAUM; AUSTIN, 2013). A Tabela 1 apresenta a definição técnica de cada registrador e seu funcionamento específico na PoC.

Sigla	Nome técnico	Largura	Escopo na PoC	Função Teórica e Operação Efetiva na PoC
PC	Program Counter (Contador de Programa)	8 bits	.circ e Web	Armazena o endereço de memória (00h–FFh) da próxima palavra a ser buscada pela CPU. Na PoC, está acoplado a um somador constante (+1); quando o sinal PC_Inc = 1 é acionado, atualiza seu estado com PC ← (PC + 1) mod 256 na borda de subida do clock.
MAR	Memory Address Register (Registrador de Endereço de Memória)	8 bits	.circ e Web	Armazena o endereço específico da posição de memória que será lida ou escrita, alimentando diretamente a porta de seleção A da RAM pelo barramento de endereços. Quando MAR_Load = 1, captura o endereço vindo do PC (MAR ← PC, ou MAR ← IR[3:0] em instruções de memória no Modo Automático).
MBR (ou MDR)	Memory Buffer Register (Memory Data Register)	8 bits	.circ e Web	Registrador de fronteira entre o barramento de dados da memória e o caminho de dados interno da CPU. Quando RAM_Read_MBR = 1 (ou RAM_Read + MBR_Load), armazena na subida do clock o byte lido de RAM[MAR], fornecendo-o simultaneamente à entrada do IR e à entrada da ULA.
IR	Instruction Register (Registrador de Instrução)	8 bits	.circ e Web	Retém a palavra de instrução recém-buscada do MBR (IR ← MBR quando IR_Load = 1) enquanto ela é inspecionada e executada. No Modo Manual, expõe o byte para interpretação pelo estudante; no Modo Automático, fornece

				IR[7:4] (opcode) e IR[3:0] (operando/endereço) para a UC.
AC	Accumulator (Acumulador)	8 bits	.circ e Web	Registrador central em arquiteturas de acumulador: fornece continuamente um dos operandos de entrada para a ULA e armazena o resultado produzido pela operação (AC ← AC + MBR) quando AC_Exec = 1 (ou AC_Load = 1), enviando seu valor também ao pino Monitor.
SAIDA	Output Register (Registrador de Saída)	8 bits	Web (Modo Auto)	Registrador dedicado que captura o valor do Acumulador (SAIDA ← AC) exclusivamente quando a instrução OUT (0x5) ativa o sinal SAIDA_Load = 1.

4.4. Barramentos de Endereços, Dados e Controle

Os barramentos (buses) são os caminhos físicos compartilhados que interligam a CPU, a memória principal e os módulos de Entrada e Saída (STALLINGS, 2010). Na PoC, os três barramentos possuem representação visual independente e podem ser filtrados individualmente na interface do simulador web:

1. **Barramento de Endereços (Address Bus — 8 bits, unidirecional)**: conjunto de 8 linhas condutoras que transmite o endereço do PC para o MAR e, a partir da saída do MAR, alimenta a porta de seleção A da memória principal (RAM 256 x 8). Sua largura de 8 bits define o espaço máximo de endereçamento de 2^8 = 256 posições (00h a FFh);

2. **Barramento de Dados (Data Bus — 8 bits)**: via de 8 bits que transporta palavras binárias entre a porta de saída D da memória RAM 256 x 8 e a entrada do registrador MBR, bifurcando-se internamente a partir da saída do MBR para alimentar a entrada do IR e a entrada da ULA, além de conduzir o resultado da ULA ao AC e do AC ao periférico de saída Monitor (bem como do pino de entrada Teclado para a escrita na memória ou MBR);

3. **Barramento de Controle (Control Bus) e Rede de CLOCK (1,0 Hz)**: conjunto de linhas dedicadas a coordenar a habilitação de escrita (Write Enable — WE), habilitação de leitura/saída (Output Enable — OE) e incremento de contador sob o sincronismo da borda de subida do sinal CLOCK. A Tabela 2 detalha todos os sinais de controle da PoC.

Sinal de Controle	Ambiente na PoC	Componente Controlado	Micro-operação Executada na Borda de Subida do CLOCK.
MAR_Load	.circ + Web (Manual/Auto)	Pino WE do MAR	Habilita a escrita do endereço no registrador MAR (MAR ← PC ou MAR ← IR[3:0]).
RAM_Read_MBR	.circ + Web (Manual/Auto)	Pino OE da RAM e WE do MBR	Habilita a saída de dados da RAM[MAR] no barramento de dados e grava o byte no MBR (MBR ← RAM[MAR]).
IR_Load	.circ + Web (Manual/Auto)	Pino WE do IR	Habilita a escrita da palavra do MBR no Registrador de Instrução (IR ← MBR).
PC_Inc	.circ + Web (Manual/Auto)	Entrada de contagem/WE do PC	Incrementa o Contador de Programa em uma unidade (PC ← PC + 1).
AC_Exec (ou AC_Load)	.circ + Web (Manual/Auto)	Pino WE do AC	Habilita a escrita no Acumulador para reter o resultado calculado pela ULA (AC ← AC + MBR ou AC ← MBR).
RAM_Write	Web (Modo Auto)	Pino WE da RAM	Grava sincronamente o valor do MBR na posição RAM[MAR] (RAM[MAR] ← MBR, instrução STA).
MBR_From_AC	Web (Modo Auto)	Entrada do MBR	Transfere o conteúdo do AC para o MBR (MBR ← AC, preparatório da instrução STA).
ALU_Op	Web (Modo Auto)	Seletor da ULA	Seleciona a operação da ULA: ADD (instruções ADDI e ADD) ou PASS (instruções LDI, LDA e IN). O operando efetivo vem do MBR nas instruções ADD e LDA; em ADDI e LDI vem de IR [3:0], e em IN vem do Teclado.
SAIDA_Load	Web (Modo Auto)	Pino WE de SAIDA	Atualiza o registrador de saída com o valor do AC (SAIDA ← AC, instrução OUT).
IN_Read	Web (Modo Auto)	Entrada do AC	Captura os 8 bits do

			dispositivo Teclado diretamente para o Acumulador (AC ← TECLADO, instrução IN).
HALT	Web (Modo Auto)	Gerador de CLOCK	Interrompe o avanço automático do ciclo ao decodificar a instrução HLT (0xF).

(Nota sobre o dispositivo Teclado: no circuito esquemático e no Modo Manual, o Teclado fornece uma palavra externa de 8 bits que pode ser gravada na posição apontada pelo MAR via interface de bancada; no Modo Automático do simulador web, o teclado é lido diretamente para o Acumulador pela instrução IN [0x6] através do sinal IN_Read [AC ← TECLADO], sem trafegar pelo registrador MBR).

4.5. Hierarquia de Memória e o Papel da Memória Cache

A organização de memória de um computador estrutura-se em níveis hierárquicos de velocidade, capacidade e custo por bit para mitigar o Memory Wall (visto na Seção 2.1), equilibrando tempo de acesso do topo até a base da pirâmide (STALLINGS, 2010; TANENBAUM; AUSTIN, 2013):

1. Registradores Internos (PC, MAR, MBR, IR, AC): localizados no interior da CPU, oferecem o acesso mais veloz (na frequência do clock, 1 ciclo), mas armazenam apenas poucos bytes simultaneamente;
2. Memória Cache (L1, L2 e L3): memória estática ultrarrápida (tecnologia SRAM) situada entre a CPU e a RAM, responsável por reter cópias dos blocos de instruções e dados mais requisitados;
3. Memória Principal (RAM): memória volátil (tecnologia DRAM em sistemas comerciais) que armazena temporariamente os programas e dados em execução, oferecendo capacidade intermediária e latência superior à da cache;
4. Armazenamento Secundário (SSD/HD): dispositivos não voláteis de grande capacidade destinados à persistência permanente de arquivos e sistemas, apresentando a maior latência da hierarquia.

O funcionamento da memória cache fundamenta-se no Princípio da Localidade de Referência (TANENBAUM; AUSTIN, 2013):

- Localidade Temporal: um endereço de memória referenciado no instante atual possui alta probabilidade de ser referenciado novamente em um futuro próximo (como instruções em um laço ou variáveis acumuladoras);
- Localidade Espacial: endereços contíguos ao endereço recém-acessado possuem alta probabilidade de serem requisitados nos ciclos seguintes (como ocorre na busca sequencial de instruções PC ← PC + 1).

Quando a CPU solicita o endereço apontado pelo MAR, o subsistema consulta primeiramente a cache. Caso o bloco esteja presente e válido (V = 1 e Tag coincidente), ocorre um acerto (cache hit) e o dado é entregue rapidamente ao MBR. Caso não esteja presente, ocorre uma falha (cache miss): o bloco precisa ser buscado na memória RAM (com penalidade de ciclos) e copiado para uma linha da cache.

Relação direta com a PoC:

- No esquemático neocorecpu.circ (e no Modo Manual estrito), estão implementados fisicamente o topo da pirâmide (Registradores Internos) e o nível de Memória Principal (RAM 256 x 8), sem bloco físico de cache no circuito lógico;
- No esquemático neocore_cpu.circ (e no Modo Manual estrito), estão implementados fisicamente o topo da pirâmide (Registradores Internos) e o nível de Memória Principal (RAM 256 x 8), sem bloco físico de cache no circuito lógico;

5. O Ciclo de Instrução (Fetch – Decode – Execute) e Caso de Teste

O ciclo de instrução compreende o processo sequencial repetido continuamente pela CPU para buscar, interpretar e executar cada operação de um programa armazenado na memória. Utilizando a Notação de Transferência entre Registradores (RTL), detalham-se a seguir as etapas fundamentais implementadas na PoC.

Convenção de notação dos instantes Tn. Neste trabalho, Tn designa um pulso de clock, e a numeração depende do modo de operação. No Modo Manual (circuito .circ e simulador web), cada instrução ocupa quatro pulsos — T0 a T3 — e a Tabela 3 apresenta a contagem contínua de dois ciclos consecutivos (T0 a T7, em que T4 a T7 repetem a sequência elementar de T0 a T3). No Modo Automático do simulador web, a máquina de estados da UC opera por instrução: todas as instruções executam a busca em T0–T2 e a decodificação em T3 (DECODE); em seguida, instruções imediatas e de E/S completam a execução em T4 (T0–T4, totalizando 5 pulsos: ADDI, LDI, OUT, IN, HLT), enquanto instruções que acessam a RAM utilizam os estados T4 a T6 (T0–T6, totalizando 7 pulsos: LDA, ADD, STA). O estado de parada HLT repousa em T4. Sempre que um instante Tn for citado, o modo correspondente é indicado ou decorre do contexto da seção.

5.1. Fase de Busca (Fetch)

Durante a fase de busca, o processador localiza e lê a próxima palavra na memória RAM 256 x 8 a partir do endereço indicado pelo registrador PC. No circuito base (neocore_cpu.circ) e no Modo Manual, o processo desenvolve-se nos instantes T0, T1 e T2:

- T0 (MAR ← PC): com o sinal MAR_Load = 1, na borda de subida do CLOCK, o endereço contido no PC é copiado para o registrador MAR, estabilizando esse endereço no barramento de endereços;
- T1 (MBR ← RAM[MAR]): com o sinal RAM_Read_MBR = 1, a memória principal é lida na posição apontada pelo MAR e o byte resultante é transferido pelo barramento de dados para dentro do registrador MBR;
- T2 (IR ← MBR; PC ← PC + 1): com os sinais IR_Load = 1 e PC_Inc = 1 acionados conjuntamente, o conteúdo do MBR é carregado no registrador de instrução IR, enquanto o contador PC é incrementado em uma unidade para apontar para o próximo endereço sequencial. (Nota: a realização simultânea da carga do IR e do incremento do PC em T2 é viável porque ambos utilizam caminhos e portas de escrita independentes, otimizando o número de ciclos).

5.2. Fase de Decodificação (Decode) e Responsabilidade na PoC

Em uma CPU completa dotada de Unidade de Controle por hardware, a etapa de decodificação consiste na análise combinacional dos bits contidos no IR para identificar a operação solicitada e preparar os sinais de habilitação da fase de execução, sem ocorrer transferência de dados entre registradores de trabalho.

Na PoC desenvolvida, a responsabilidade pela execução do Decode é explicitada de acordo com o modo de operação:

1. No circuito neocore_cpu.circ e no Modo Manual do simulador web: como o protótipo esquemático não particiona o byte em opcode por hardware nem dispõe de decodificador lógico automático, o próprio estudante executa cognitivamente a etapa de Decode ao término de T2: ele inspeciona o valor carregado no IR e decide acionar manualmente o pino AC_Exec para comandar a adição na etapa seguinte;
2. No Modo Automático do simulador web: a máquina de estados finitos da UC inclui um ciclo específico para decodificação (estado T3 · DECODE, na numeração própria do Modo Automático), no qual o decodificador em software separa IR[7:4] (opcode de 4 bits: 0x0 ADDI, 0x1 LDI, 0x2 LDA, 0x3 ADD, 0x4 STA, 0x5 OUT, 0x6 IN, 0xF HLT) e IR[3:0] (operando/endereço de 4 bits), selecionando automaticamente os sinais de controle para os pulsos de execução subsequentes.

5.3. Fase de Execução (Execute)

Na fase de execução, os sinais de controle acionam as unidades funcionais pertinentes para concretizar a operação. Na operação de adição acumulada da PoC, a ULA soma o valor contido no Acumulador (AC) com o operando retido no MBR e armazena o resultado de volta no próprio acumulador:

- T3 — Modo Manual (AC ← AC + MBR): com o sinal AC_Exec = 1, o resultado de 8 bits presente na saída da ULA é gravado no registrador AC na borda de subida do CLOCK e exibido simultaneamente no dispositivo Monitor.

5.4. Caso de Teste Passo a Passo e Tabela de Trace (Soma $5 + 3 = 8$)

Para validar experimentalmente o funcionamento coordenado das transferências entre registradores no cenário de referência do projeto (Soma Clássica SAP-1: 5 + 3 = 8), os primeiros endereços da memória principal são inicializados com RAM[00h] = 05h (5 dec) e RAM[01h] = 03h (3 dec), iniciando todos os registradores (PC, MAR, MBR, IR e AC) com o valor 00h:

- Passo 1 — Busca / Fetch Parte 1 (T0: MAR ← PC): Ativa-se MAR_Load = 1 e aplica-se um pulso de CLOCK. O endereço 00h contido no PC é copiado para o MAR, selecionando a posição 00h da memória RAM pelo barramento de endereços.
- Passo 2 — Busca / Fetch Parte 2 (T1: MBR ← RAM[MAR]): Desativa-se MAR_Load = 0 e ativa-se RAM_Read_MBR = 1. No pulso de CLOCK, a RAM disponibiliza o byte 05h no barramento de dados e o registrador MBR armazena esse valor (MBR = 05h).
- Passo 3 — Busca / Fetch Parte 3: Carga do IR e Avanço do PC (T2: IR ← MBR; PC ← PC + 1): Desativa-se RAM_Read_MBR = 0 e ativam-se simultaneamente IR_Load = 1 e PC_Inc = 1. Com o pulso de CLOCK, a palavra 05h (00000101 em binário) é transferida para o IR e o contador PC avança para 01h. No Modo Manual, a decodificação é realizada cognitivamente pelo estudante após o término de T2, sem um pulso de clock próprio: o estudante lê visualmente o conteúdo do IR e decide acionar manualmente o pino AC_Exec no passo subsequente (T3).
- Passo 4 — Execução / Execute (T3: AC ← AC + MBR): Desativam-se os sinais anteriores e ativa-se AC_Exec = 1. O somador da ULA realiza a adição entre AC (00h) e MBR (05h). Na subida do CLOCK, o resultado 05h é gravado no AC e exibido no pino Monitor como 00000101 (5 dec).
- Passo 5 — Segundo Ciclo Completo (T4 a T7): Repete-se a sequência de quatro micro-operações para o endereço 01h agora apontado pelo PC:
  - Em T4 (MAR ← PC), com MAR_Load = 1, o MAR recebe 01h;
  - Em T5 (MBR ← RAM[01h]), com RAM_Read_MBR = 1, o MBR recebe o segundo byte da memória (03h);
  - Em T6 (IR ← MBR; PC ← PC + 1), com IR_Load = 1 e PC_Inc = 1, o IR atualiza-se para 03h e o PC avança para 02h;
  - Em T7 (AC ← AC + MBR), com AC_Exec = 1, a ULA soma o valor acumulado no AC (05h) ao novo operando no MBR (03h), gravando no AC o resultado final 08h (8 dec) e exibindo 00001000 no pino Monitor.

A Tabela 3 consolida o trace completo de estados dos registradores, sinais de controle e saída do Monitor do início ao pulso T7.

Passo (Clock)	Fase do Ciclo	Sinais de Controle Ativos	Micro-operação (Notação RTL)	PC	MAR	MBR	IR	AC	Monitor (binário / dec)
Início	Estado Inicial (Reset)	Nenhum (00000)	Estado Inicial (Reset)	00h	00h	00h	00h	00h	00000000 (0)
T0	1º Ciclo — Fetch (Endereço)	MAR_Load = 1	MAR ← PC	00h	00h	00h	00h	00h	00000000 (0)
T1	1º Ciclo — Fetch (Memória)	RAM_Read_MBR = 1	MBR ← RAM[00h]	00h	00h	05h	00h	00h	00000000 (0)
T2	1º Ciclo — Fetch (carga do IR e avanço do PC)	IR_Load = 1, PC_Inc = 1	IR ← MBR; PC ← PC + 1	01h	00h	05h	05h	00h	00000000 (0)
T3	1º Ciclo — Execute (Soma)	AC_Exec = 1	AC ← AC + MBR (0+5)	01h	00h	05h	05h	05h	00000101 (5)
T4	2º Ciclo — Fetch (Endereço)	MAR_Load = 1	MAR ← PC	01h	01h	05h	05h	05h	00000101 (5)
T5	2º Ciclo — Fetch (Memória)	RAM_Read_MBR = 1	MBR ← RAM[01h]	01h	01h	03h	05h	05h	00000101 (5)
T6	2º Ciclo — Fetch (carga do IR e avanço do PC)	IR_Load = 1, PC_Inc = 1	IR ← MBR; PC ← PC + 1	02h	01h	03h	03h	05h	00000101 (5)
T7	2º Ciclo — Execute (Soma)	AC_Exec = 1	AC ← AC + MBR (5+3)	02h	01h	03h	03h	08h	00001000 (8)

(Nota sobre a Cache L1 no Modo Automático — Demo 1): No Modo Manual ilustrado na Tabela 3, o acesso à RAM ocorre de forma direta, sem acesso à cache, refletindo a ausência de cache física no esquemático de bancada. O subsistema de Cache L1 atua no Modo Automático do simulador web, cujos estados T são contados por instrução. Ao executar a

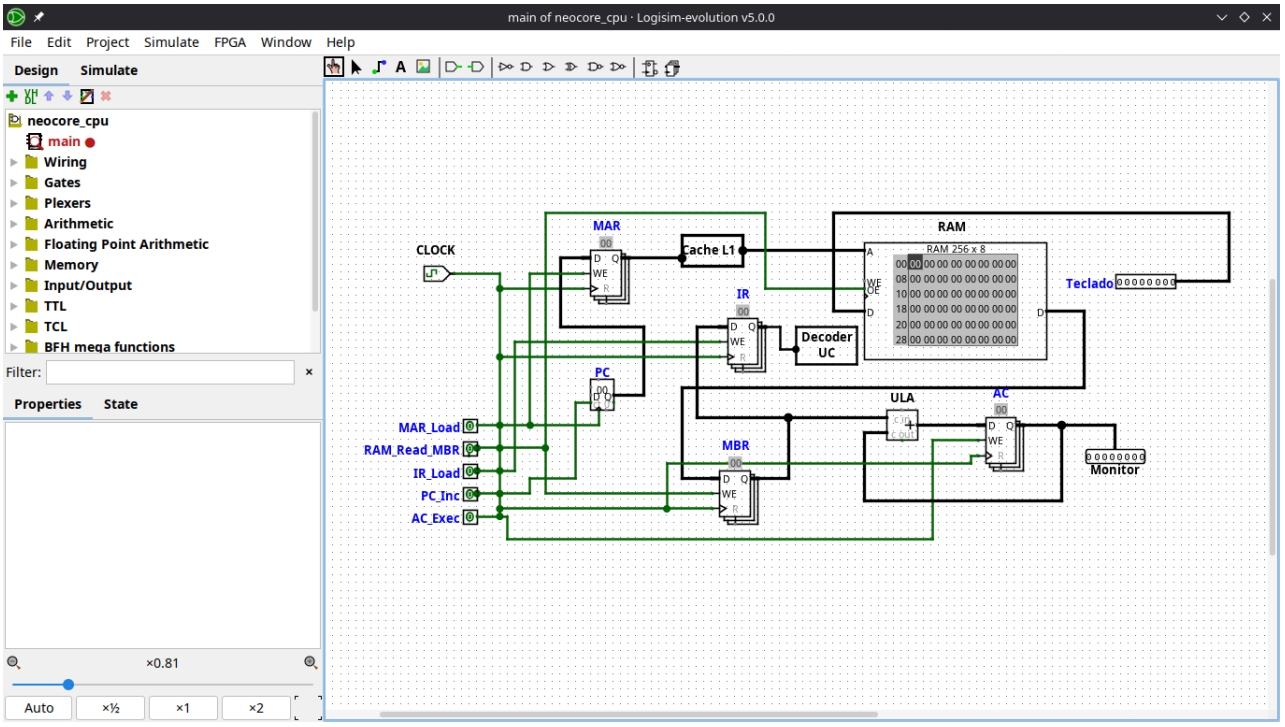
Demo 1 (RAM[00h..03h] = 05h, 03h, 50h, F0h), no pulso T1 da 1ª instrução (ADDI 5, busca em 00h), ocorre um Cache Miss (custo de 4 ciclos), carregando a linha L0 com o bloco de 2 bytes contíguos [05h, 03h]. No pulso T1 da 2ª instrução (ADDI 3, busca em 01h), a CPU localiza a palavra previamente carregada na linha L0 (V = 1, Tag = 00h, Offset = 1), registrando um Cache Hit por localidade espacial com custo de apenas 1 ciclo.

6. Modelagem e Implementação da Parte Prática

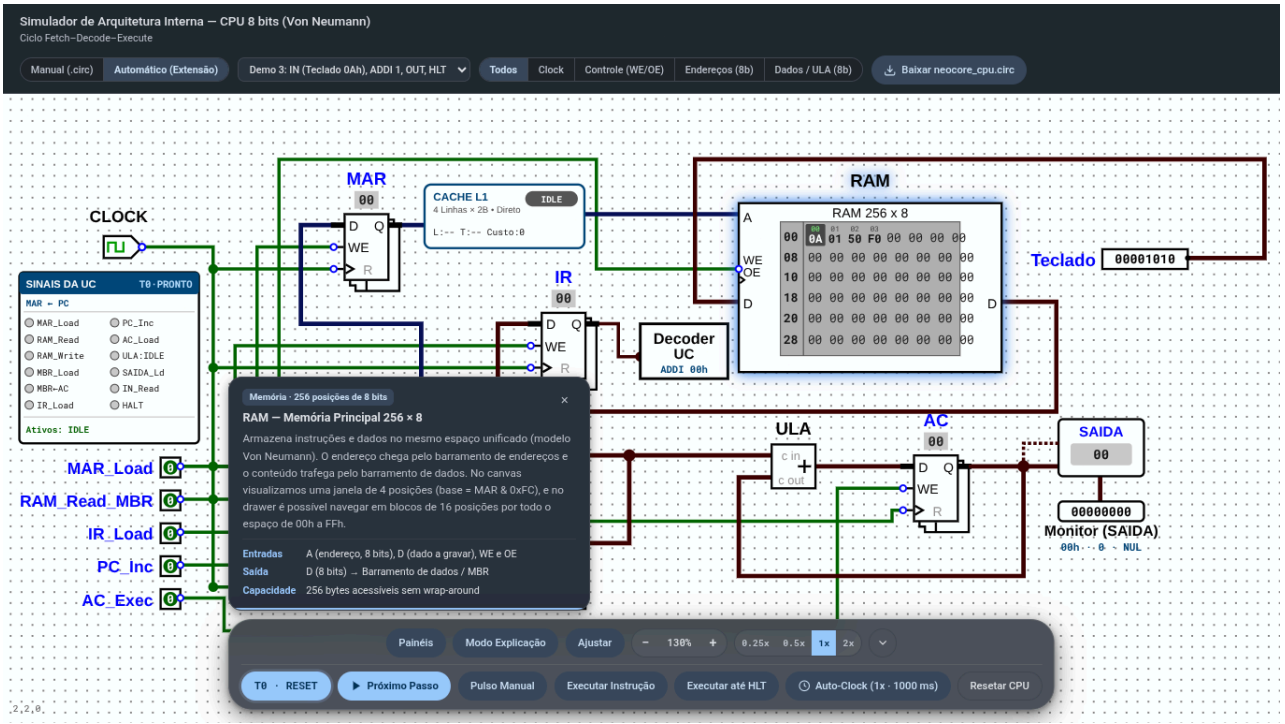
Para materializar de forma visual e interativa os conceitos de caminho de dados, barramentos e transferências entre registradores, a Parte Prática foi implementada combinando a simulação lógica de hardware e uma aplicação web dedicada.

6.1. Ferramentas Utilizadas e Arquitetura do Projeto

- **Logisim-Evolution** (v5.0.0): utilizado para modelar e validar o esquemático lógico digital (src/neocore_cpu.circ), operando com barramentos de 8 bits e gerador de clock calibrado em 1,0 Hz;
- **Simulador Web Interativo** (index.html, src/styles.css e src/script.js): desenvolvido em HTML5, CSS3 (Material Design 3) e JavaScript puro (Vanilla ES6+), sem bibliotecas externas (apenas a fonte Roboto, carregada do Google Fonts).
A aplicação interativa hospedada no GitHub: https://joaopedro-chaves.github.io/simulacao_analise_arquitetura_interna_computador_projeto_UniFECAF/, e o **repositório oficial público do projeto**: https://github.com/joaopedro-chaves/simulacao_analise_arquitetura_interna_computador_projeto_UniFECAF. O código-fonte separa rigorosamente o Modelo Lógico Puro (delimitado entre /* MODEL:START */ e /* MODEL:END */ em src/script.js) da camada de interface e renderização SVG, sendo validado por uma suíte automatizada de 28 testes unitários em Node.js (test_model.js).



Legenda: Esquemático completo do circuito digital da CPU NeoCore 8-bit modelado no ambiente Logisim-Evolution (neocore_cpu.circ).



Legenda:Visão geral da interface do Simulador Web Interativo em Material Design 3 com o diagrama esquemático SVG e a barra de controle inferior (Dock).

6.2. Estrutura do Circuito e Fluxo de Dados

O circuito prático integra os registradores da CPU, a memória principal e os pinos de Entrada e Saída por meio de quatro redes fundamentais, distribuídas espacialmente da esquerda (endereçamento e memória) para o centro (transferência e decodificação) e para a direita (processamento aritmético e saída):

1. Rede de Clock (1 bit): origina-se no gerador de pulsos (CLOCK) e distribui o sinal de sincronismo para as entradas de relógio (CLK) dos registradores PC, MAR, IR, MBR e AC (a memória RAM é de leitura combinacional e escrita controlada por WE, não recebendo a linha da rede de clock dos registradores; o registrador SAIDA pertence à extensão web e não possui linha própria de clock no SVG). Todas as transferências ocorrem de forma síncrona na borda de subida do clock;

2. Linhas de Controle do Datapath: compreendem os 5 pinos de acionamento manual (MAR_Load, RAM_Read_MBR, IR_Load, PC_Inc e AC_Exec) presentes no .circ e no Modo Manual, além do mini-HUD de 12 LEDs comandado pela UC_TABLE no Modo Automático do simulador web;

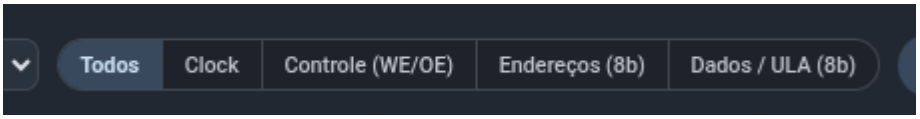
3. Caminho e Barramento de Endereços (8 bits, unidirecional): conecta a saída do contador PC à entrada de dados do MAR e, a partir da saída do MAR, alimenta a porta de endereçamento A da memória principal (RAM 256 x 8) e da Cache L1;

4. Barramento de Dados (8 bits) e Pinos de E/S: transporta o byte lido da porta D da RAM até a entrada do MBR. A partir da saída do MBR, o barramento bifurca-se para alimentar a entrada do IR (acima do bloco Decoder UC) e a entrada A (superior) da ULA (Adder de 8 bits). A saída da ULA alimenta o registrador AC, cuja saída realimenta a entrada B (inferior) da ULA e é exibida em tempo real no pino de saída Monitor (8 bits) e no registrador SAIDA (bem como do pino de entrada Teclado para escrita na RAM ou diretamente para o AC na instrução IN).

6.3. Recursos Didáticos Interativos do Simulador Web

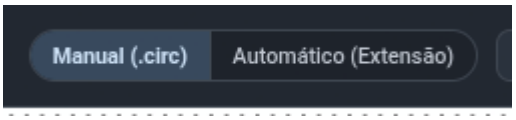
O simulador web expande a experiência laboratorial para os estudantes da NeoCore Tech por meio de seis funcionalidades didáticas:

1. Destaque dinâmico de barramentos e filtros de camadas: permite isolar visualmente as trilhas de Clock, Controle, Endereço e Dados, iluminando com destaque dos fios ativos (mudança de cor e espessura das linhas) os caminhos acionados em cada pulso;



Legenda: Filtros de camadas e destaque visual dinâmico das trilhas ativas de barramento (dados, endereços, controle e clock).

2. Alternância entre Modo Manual (SAP-1) e Modo Automático (UC + ISA): permite que o aluno execute primeiramente o ciclo controlando manualmente os 5 pinos (Demo 0) e, em seguida, teste programas completos (Demos 1 a 3) com decodificação automática de opcodes (LDI, ADDI, LDA, ADD, STA, OUT, IN, HLT);



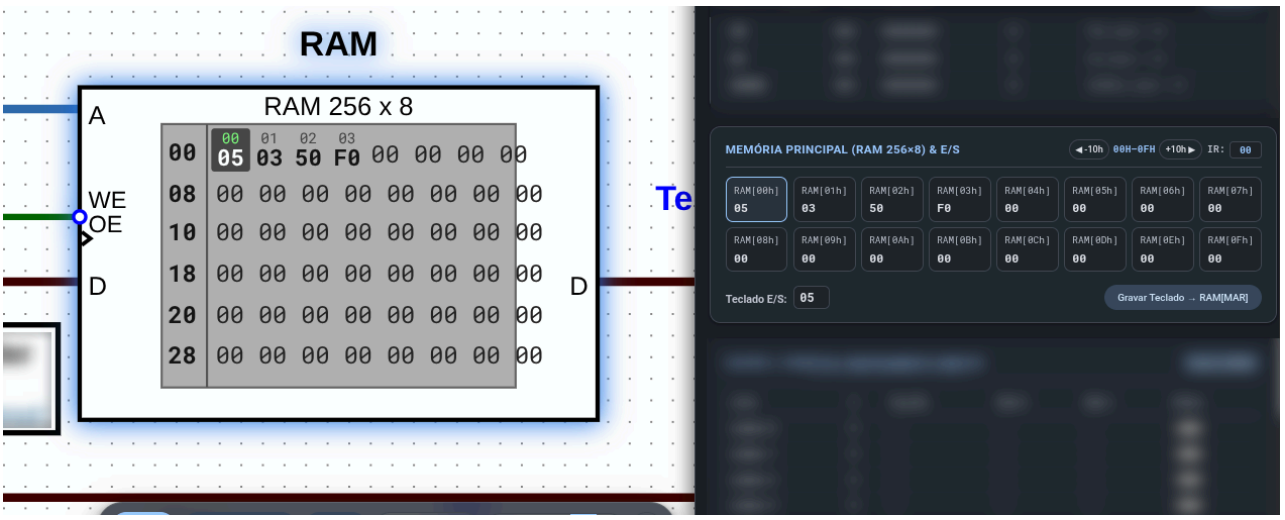
Legenda da Figura 5: Seletor de modo de operação e painel de execução dos programas de demonstração (Modo Manual vs. Modo Automático).

3. Inspeção multi-base simultânea (HEX, BIN e DEC): exibe o valor de todos os registradores e do Monitor simultaneamente nas bases hexadecimal, binária de 8 bits e decimal sem sinal (e caractere ASCII no Monitor);



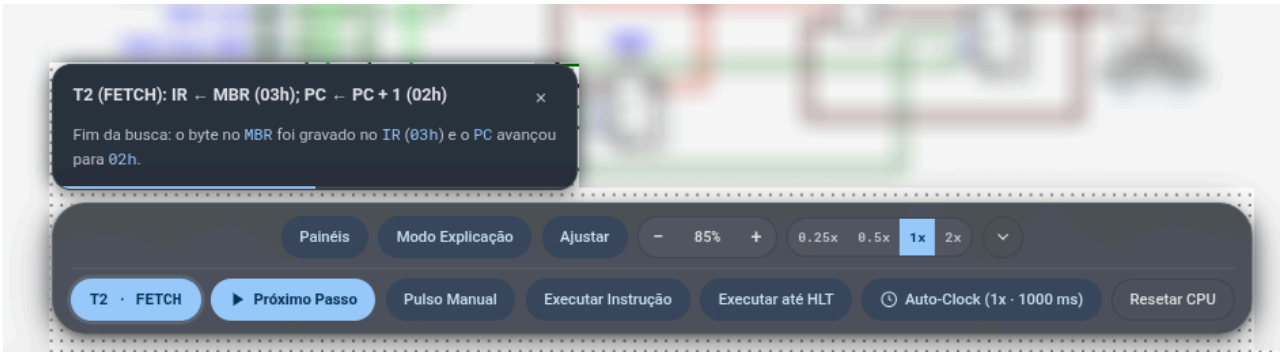
Legenda: Painel de inspeção multi-base simultânea dos registradores internos e periféricos de E/S em formato Hexadecimal, Binário e Decimal.

4. Editor de RAM paginado (00h–FFh) e monitor de Cache L1: possibilita navegar pelas 16 páginas de 16 bytes das 256 posições de memória, editar valores em hexadecimal e observar os contadores de Cache Hit, Cache Miss e ciclos de latência;



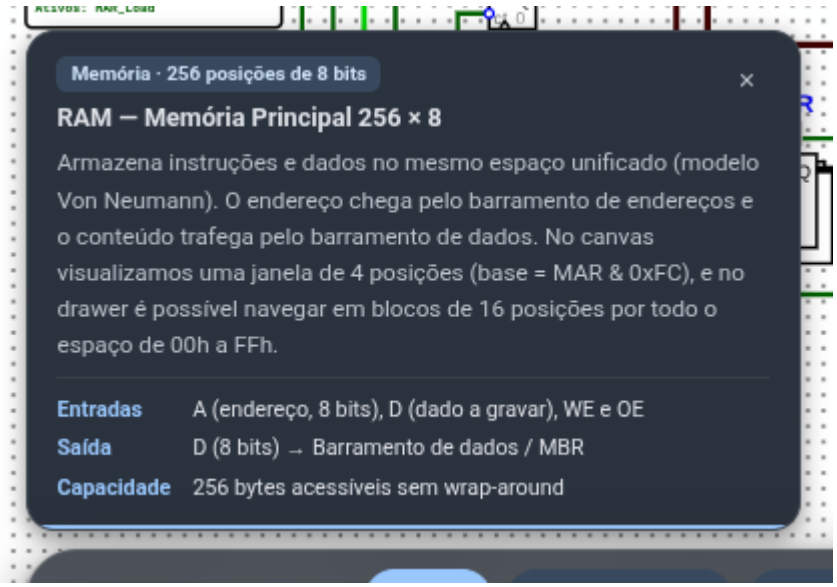
Legenda: Editor interativo de memória RAM paginado (00h a FFh) e monitor em tempo real das métricas da Cache L1 didática.

5. Guia passo a passo sincronizado e histórico RTL (Dock): registra cada transferência entre registradores executada (T0 a T7), permitindo avançar pulso a pulso, saltar para qualquer etapa ou acionar o Auto-Clock com velocidades ajustáveis de 0,25x, 0,5x, 1x e 2x;



Legenda: Guia de execução passo a passo do ciclo de instrução, seletor de velocidade do Auto-Clock e histórico de micro-operações RTL.

6. Cartões contextuais de explicação (ctx-card): ao ativar o botão Modo Explicação e tocar em qualquer componente do diagrama SVG, exibe a explicação técnica e o papel prático daquele elemento no circuito.



Legenda: Cartão contextual de explicação técnica (Modo Explicação) ativado na inspeção direta dos componentes da CPU.

Adicionalmente, a interface disponibiliza controle interativo para ligar/desligar a Cache L1 (operando sob políticas Write-Through e No-Write-Allocate) e botão dedicado para copiar o histórico de execução (log RTL) em formato Markdown para relatórios e auditorias.

7. Conclusões e Limitações

7.1. Síntese Coordenada do Ciclo de Instrução e Resposta

O desenvolvimento do circuito digital no Logisim-Evolution, aliado ao simulador web interativo, cumpriu o propósito pedagógico delineado junto à NeoCore Tech: transformar o estudo das transferências entre registradores em uma experiência visual, auditável e manipulável.

Respondendo diretamente à questão central do projeto — "Como os diferentes componentes de um computador trabalham de forma coordenada para buscar, interpretar e executar uma instrução?" —, a simulação demonstra que o processamento computacional decorre de uma cadeia de eventos estritamente sincronizada pelo sinal de clock:

- Busca coordenada (Fetch): o Contador de Programa (PC) indica a referência espacial da instrução atual e transfere esse endereço ao Registrador de Endereço de Memória (MAR), que o estabiliza no barramento de endereços (MAR ← PC). A hierarquia de memória (Cache L1 e RAM 256 x 8) localiza a posição selecionada e entrega a palavra de 8 bits ao Registrador de Buffer de Memória (MBR) através do barramento de dados (MBR ←

RAM[MAR]);
- Interpretação (Decode): o Registrador de Instrução (IR) retém a palavra recebida (IR ← MBR) enquanto o PC avança em paralelo (PC ← PC + 1). Em seguida, a Unidade de Controle — exercida cognitivamente pelo estudante no Modo Manual do circuito .circ ou pela máquina de estados finitos no Modo Automático do simulador web — interpreta o conteúdo do IR e seleciona quais linhas do barramento de controle devem ser acionadas;
- Execução e saída (Execute): a Unidade Lógica e Aritmética (ULA) recebe os operandos vindos do Acumulador (AC) e do MBR, realiza a soma binária de 8 bits e, mediante o pulso no sinal de controle de execução (AC_Exec), grava o resultado de volta no AC (AC ← AC + MBR) para exibição imediata no periférico de saída (Monitor / SAIDA).

7.2. Limitações da Prova de Conceito (PoC)

Em conformidade com o rigor técnico do projeto, registram-se as limitações deliberadas do recorte implementado:

1. No esquemático neocore_cpu.circ (Logisim-Evolution): o circuito opera exclusivamente com controle manual de 5 pinos e trata os bytes lidos da memória como operandos diretos do somador fixo de 8 bits da ULA (AC + MBR), não dispondo em hardware de decodificador automático de opcodes, operações de subtração/lógica booleana, registrador de flags de estado (Zero, Carry), desvios condicionais (JMP, JZ) ou bloco físico de memória cache;
2. Na extensão do Simulador Web (index.html / src/script.js): embora o Modo Automático implemente UC automática por máquina de estados, ISA de 8 bits e Cache L1 de 4 linhas, o formato de instrução de 1 byte (4 bits de opcode + 4 bits de operando/endereço) limita o endereçamento direto por instrução às 16 primeiras células da RAM (00h a 0Fh), além de operar de forma escalar multiciclo sem pipelining.

8. Referências

CHAVES, João Pedro Vieira da Costa. Simulação e Análise de Arquitetura Interna de um Computador — NeoCore 8-bit CPU (Repositório Oficial do Projeto). GitHub Repository, 2026. Disponível em: https://github.com/joaopedro-chaves/simulacao_analise_arquitetura_interna_computador_projeto_UniFECAF.

CHAVES, João Pedro Vieira da Costa. Simulador Web Interativo — NeoCore 8-bit CPU. GitHub Pages, 2026. Disponível em: https://joaopedro-chaves.github.io/simulacao_analise_arquitetura_interna_computador_projeto_UniFECAF/.

LOGISIM-EVOLUTION DEVELOPMENT TEAM. Logisim-Evolution: Digital Logic Design Tool and Simulator (v5.0.0). GitHub Repository, 2024. Disponível em: https://github.com/logisim-evolution/logisim-evolution. Acesso em: 29 out. 2026.

MALVINO, Albert Paul; BROWN, Jerald A. Digital computer electronics. 3. ed. New York: Glencoe/McGraw-Hill, 1992.

PATTERSON, David A.; HENNESSY, John L. Organização e projeto de computadores: a interface hardware/software. 5. ed. Rio de Janeiro: Elsevier, 2017.

STALLINGS, William. Arquitetura e organização de computadores. 8. ed. São Paulo: Pearson Prentice Hall, 2010.

TANENBAUM, Andrew S.; AUSTIN, Todd. Organização estruturada de computadores. Tradução: Daniel Vieira; revisão técnica: Wagner Luiz Zucchi. 6. ed. São Paulo: Pearson Prentice Hall, 2013.

TANENBAUM, Andrew S.; BOS, Herbert. Sistemas operacionais modernos. Tradução: Jorge Ritter; revisão técnica: Raphael Y. de Camargo. 4. ed. São Paulo: Pearson Education do Brasil, 2016.